

Integration One-Pager

Uxify finds friction. BanditDB removes it — learning the best decision per user, in real time.

Uxify = eyes. BanditDB = reflexes.

The gap

Uxify excels at **detection** (Reality, Experience, Engagement) and ships heuristic **optimizers** (Navigation AI prerender, INProve). What's missing is a closed online-learning loop that *chooses the best variant per user context and provably converges*. That is BanditDB.

A contextual bandit beats classic A/B testing: no fixed traffic split, it routes traffic to the winning variant **while still exploring**, and adapts to drift (seasonality, trends, new inventory).

Where BanditDB plugs in

UXIFY PRODUCT	BANDITDB ROLE	ARMS	CONTEXT	REWARD
Navigation AI	Prerender target picker	candidate next pages	device, referrer, page depth, viewport	click + low latency
CRO	Variant allocation engine	layout / CTA / copy variants	segment, geo, device, source	add-to-cart / purchase
Publishers / Media	Engagement maximizer	headline, thumbnail, recirc module, ad slot	topic, source, time-of-day	pageview depth + ad click
INProve	Perf-strategy selector	lazy-load / image tier / defer policy	connection type, device class	INP/LCP gain w/o conv loss
Ask Uxi AI	Decision backend (MCP)	any campaign	NL query → context	logged outcomes

Model menu — which BanditDB algorithm for which Uxify job

Pick per surface by **context shape** (linear vs nonlinear) and **exploration need**.

BANDITDB MODEL	WHEN TO REACH FOR IT	BEST UXIFY USE CASE
LinUCB	Linear context→reward, want tight regret bound + deterministic, explainable scores	Navigation AI prerender pick: small clean feature set (device, depth, referrer), needs auditable choices
Thompson Sampling	Linear context, faster early exploration, many parallel low-traffic arms	CRO CTA/layout test: posterior sampling explores variants fast, smooth allocation, no UCB tuning
NeuralLinUCB	Context→reward nonlinear , rich/high-dim features (embeddings), still want UCB confidence	Publishers/Media recirculation + headline: content/text embeddings as context, MLP learns nonlinear topic×reader interactions, UCB head keeps principled exploration
NeuralThompsonSampling	Nonlinear features and heavy exploration (cold start, huge/changing arm set, sparse reward)	Ecommerce product/personalization with cold-start SKUs + sparse purchase signal: neural feature net + posterior sampling explores new inventory without starving winners
Progressive Tournament	Wrap <i>any</i> base model for safe production rollout of a challenger	Champion/challenger on every surface: new variant earns traffic only after required_wins , capped 90% — bad variant can't tank revenue

Rule of thumb:

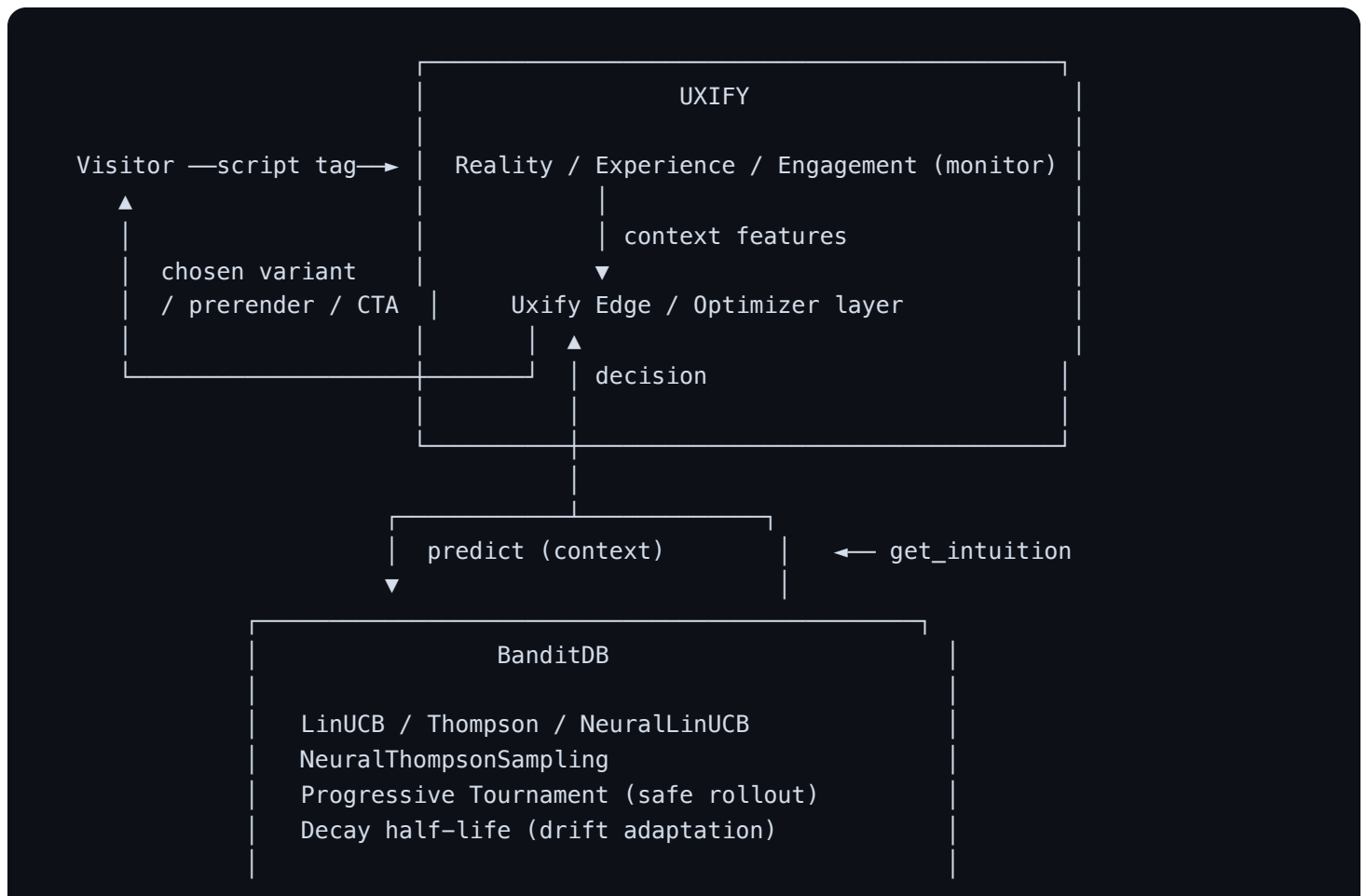
- Few clean features + need explainability → **LinUCB**.
- Few features + fast explore → **Thompson Sampling**.
- Embeddings / nonlinear → go neural.
- Among neural: confidence-driven, more stable → **NeuralLinUCB**; exploration-hungry, cold-start, sparse reward → **NeuralThompsonSampling**.

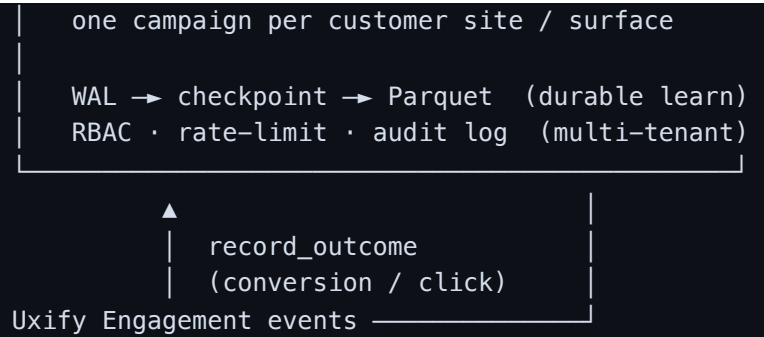
- Shipping a new variant to real traffic → always wrap in **Progressive Tournament**, layer **decay half-life** for drift.

Per-surface picks

- Navigation AI** — LinUCB (clean features, auditable). Upgrade to NeuralLinUCB if adding session-sequence embeddings.
- CRO** — Thompson Sampling baseline; NeuralThompsonSampling when personalizing per-user with rich profile vectors + sparse conversion.
- Publishers/Media** — NeuralLinUCB (content embeddings, nonlinear topicxreader).
- INProve perf** — LinUCB (small device/connection feature set, want explainable trade-off).
- Personalization / cold-start catalog** — NeuralThompsonSampling.
- All of the above in prod** — Progressive Tournament + decay half-life.

Architecture





Loop: Uxify captures context → BanditDB `predict` returns variant → Uxify renders it → Uxify Engagement observes conversion → BanditDB `record_outcome` updates. Closed, online, per-request.

Why BanditDB specifically (vs roll-your-own)

- **In-memory, low-latency** — fits edge / script-tag budget (Uxify's <10-min-integration ethos).
- **WAL durability + checkpoint** — survives restart, zero lost learning.
- **Progressive Tournament** — challenger variant earns traffic only after `required_wins`, capped at 90% (`BPS_CEIL`). Bad variant can't tank revenue.
- **Decay half-life** — forgets stale data; adapts to seasonality (holiday vs normal).
- **Per-campaign isolation** — one campaign per customer site → clean multi-tenant SaaS model.
- **SNIPS off-policy eval** — score a new variant on logged data *before* going live.
- **RBAC + rate limit + audit** — enterprise / multi-tenant ready out of the box.
- **MCP server** — `create_campaign`, `get_intuition`, `record_outcome` already exposed → Ask Uxi AI calls BanditDB as an agent-native decision backend, no glue code.

Integration modes (pick per deployment)

1. **REST sidecar** — Uxify edge calls `POST /predict` + `POST /reward`. Simplest.
2. **MCP backend** — Ask Uxi AI agent invokes `mcp_banditdb_*` tools directly.
3. **Embedded** — BanditDB co-located at Uxify edge POPs for sub-ms decisions.

30-day pilot proposal

- **Surface:** one CRO use case (CTA or layout) on one ecommerce customer.
- **Setup:** one campaign, Thompson Sampling, 3–5 arms, segment context.
- **Guardrail:** Progressive Tournament + decay; SNIPS pre-check each new variant.
- **Success metric:** add-to-cart / purchase lift vs Uxify's current A/B baseline.
- **Effort:** script already emits context + conversion events → wire two HTTP calls.